

Лекция по Java: Основные моменты

1 Знакомство с java

1. История и разработка

Java был разработан компанией **Sun Microsystems** в начале 1990-х годов. Основной идеей было создание языка, который был бы **независим от платформы**, то есть программы, написанные на Java, могли бы работать на любой операционной системе без необходимости изменения кода. Это стало возможным благодаря **Java Virtual Machine (JVM)**, которая интерпретирует байт-код Java и выполняет его на конкретной платформе.

В 2010 году Sun Microsystems была приобретена компанией **Oracle**, которая продолжает развивать и поддерживать Java.

2. Платформенная независимость

Платформенная независимость — это одно из ключевых преимуществ Java.

Программа, написанная на Java, компилируется в **байт-код**, который не зависит от конкретной операционной системы. Этот байт-код затем выполняется на **JVM**, которая существует для множества платформ, включая Windows, macOS, Linux, Android и другие.

Преимущества платформенной независимости:

- **Переносимость:** Программы могут быть легко перенесены на разные платформы без изменения исходного кода.
- **Единый код:** Разработчики пишут код один раз, и он работает на всех поддерживаемых платформах.
- **Универсальность:** Java используется для разработки широкого спектра приложений, от мобильных до корпоративных систем.

3. Объектно-ориентированный подход

Java — это **объектно-ориентированный язык программирования (ООП)**, что означает, что программы на Java строятся вокруг **объектов** и **классов**.

Основные концепции ООП в Java:

- **Класс:** Шаблон или описание для создания объектов. Класс определяет свойства (переменные) и поведение (методы) объекта.
- **Объект:** Экземпляр класса. Объекты взаимодействуют друг с другом через методы.
- **Наследование:** Возможность создания нового класса на основе существующего, наследуя его свойства и методы.
- **Инкапсуляция:** Скрытие внутренних деталей реализации и предоставление доступа только через публичные методы.
- **Полиморфизм:** Возможность использовать один и тот же интерфейс для разных типов данных.

4. Применение Java

Java широко используется в различных областях:

- **Мобильная разработка:** Java является основным языком для разработки приложений под Android.
- **Веб-разработка:** Java используется для создания веб-приложений с использованием фреймворков, таких как Spring и JavaServer Faces (JSF).
- **Корпоративные системы:** Java подходит для разработки масштабируемых и надежных корпоративных приложений.
- **Десктопные приложения:** Java используется для создания приложений для настольных компьютеров.

- **Игры и мультимедиа:** Java поддерживает разработку игр и мультимедийных приложений.

5. Сообщество и экосистема

Java имеет огромное и активное сообщество разработчиков по всему миру. Это сообщество создает множество библиотек, фреймворков и инструментов, которые расширяют возможности Java и упрощают разработку.

Некоторые популярные инструменты и фреймворки:

- **Spring Framework:** Фреймворк для создания веб-приложений и микросервисов.
- **Hibernate:** ORM-фреймворк для работы с базами данных.
- **Maven и Gradle:** Системы автоматизации сборки проектов.
- **JUnit:** Фреймворк для модульного тестирования.

6. Структура программы на Java

Пример программы "Hello, World!":

```
class MyClass {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Основные элементы:

- **Класс:** Шаблон для создания объектов. Название класса должно совпадать с именем файла (например, MyClass.java).
- **Метод main:** Точка входа в программу. Обязателен для исполняемых программ.
- **Объект:** Экземпляр класса.
- **Переменные экземпляра:** Характеристики объекта.

7. Метод main

Синтаксис:

```
public static void main(String[] args)
```

Ключевые слова:

- **public:** Доступ без ограничений.
- **static:** Метод может быть вызван без создания экземпляра класса.
- **void:** Метод не возвращает значения.
- **main:** Имя метода.
- **String[] args:** Массив строк, передаваемых в метод.

8. Вывод на консоль

Команды:

- `System.out.println():` Выводит текст и переносит курсор на новую строку.
- `System.out.print():` Выводит текст без переноса строки.

Пример:

```
System.out.println("Hello World!");
```

9. Точка с запятой

Правило:

- Каждая команда в Java должна заканчиваться точкой с запятой (;).
- Исключение: фигурные скобки {} не заканчиваются точкой с запятой.

2 Переменные в Java

1. Основные понятия

Переменные — это контейнеры для хранения данных, которые могут быть использованы и изменены в процессе выполнения программы. Каждая переменная имеет:

- **Имя (идентификатор):** Уникальное имя, которое используется для обращения к переменной.
- **Тип:** Определяет, какие данные могут храниться в переменной.
- **Значение:** Данные, которые хранятся в переменной.

2. Типы данных

В Java существует множество типов данных, но основные из них:

- **Целые числа:**
 - `int`: Для хранения целых чисел, например, 123, -456.
 - `long`: Для хранения больших целых чисел.
 - `short`: Для хранения маленьких целых чисел.
 - `byte`: Для хранения очень маленьких целых чисел.
- **Вещественные числа:**
 - `float`: Для хранения чисел с плавающей запятой, например, 3.1416.
 - `double`: Для хранения чисел с плавающей запятой с двойной точностью, например, -55.66.
- **Строки:**
 - `String`: Для хранения текстовых данных, например, "Привет", "Доброе утро!". Строки заключаются в двойные кавычки.
- **Символы:**
 - `char`: Для хранения одного символа, например, 'A', 'Z'. Символы заключаются в одинарные кавычки.
- **Логические значения:**
 - `boolean`: Для хранения логических значений `true` или `false`.

3. Объявление и инициализация переменных

Объявление переменной:

```
String name;
```

Инициализация переменной:

```
name = "Ivan";
```

Объявление и инициализация в одной строке:

```
String name = "Ivan";
int age = 25;
double score = 15.9;
char group = 'Z';
boolean atHome = true;
```

4. Примеры использования переменных

Объявление нескольких переменных одного типа:

```
int a = 42, b = 11;
```

Вывод значений переменных:

```
int x = 42;
String y = "Значение переменной x равно ";
int z = 58;

System.out.println(x);           // 42
System.out.println(x + z);       // 100
```

```
System.out.println("x = " + 42);           // x = 42
System.out.println(y + x);                 // Значение переменной x
равно 42
System.out.println(y + x + z);             // Значение переменной x
равно 4258
System.out.println(y + (x + z));           // Значение переменной x
равно 100
```

Вывод нескольких переменных в одну строку:

```
int x = 42;
int y = 84;
System.out.println(x + " " + y);           // 42 84
```

5. Важные моменты

- **Типизация:** В Java переменные строго типизированы, то есть тип переменной определяется при её объявлении и не может быть изменен.
- **Инициализация:** Переменные должны быть инициализированы (т.е. им должно быть присвоено значение) перед использованием, иначе программа выдаст ошибку.
- **Конкатенация строк:** При использовании оператора + между строкой и числом, результат будет строкой. Для выполнения арифметических операций необходимо использовать скобки.

3 Ввод данных в программу на Java

1. Использование класса Scanner

Класс Scanner — это один из наиболее распространенных способов ввода данных в программу на Java. Он позволяет считывать данные различных типов из стандартного ввода (System.in).

Импорт класса Scanner:

```
import java.util.Scanner;
```

Создание экземпляра класса Scanner:

```
Scanner sc = new Scanner(System.in);
```

2. Основные методы класса Scanner

Класс Scanner предоставляет методы для считывания различных типов данных:

- **Считывание целого числа:**

- `int x = sc.nextInt();`

- **Считывание числа с плавающей запятой:**

- `float f = sc.nextFloat();`

- **Считывание вещественного числа с двойной точностью:**

- `double d = sc.nextDouble();`

- **Считывание логического значения:**

- `boolean b = sc.nextBoolean();`

- **Считывание строки целиком:**

- `String s = sc.nextLine();`

- **Считывание одного слова (до первого пробела):**

- `String word = sc.next();`

3. Пример программы с использованием Scanner

Программа, которая считывает строку, введенную пользователем:

```
import java.util.Scanner;

class MyClass {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Введите строку:");
        String input = sc.nextLine();
        System.out.println("Вы ввели: " + input);
        sc.close();
    }
}
```

Программа, которая считывает несколько типов данных:

```
import java.util.Scanner;

class MyClass {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Введите целое число:");
        int x = sc.nextInt();

        System.out.println("Введите вещественное число:");
        double y = sc.nextDouble();
    }
}
```

```

        System.out.println("Введите строку:");
        sc.nextLine(); // Считываем символ новой строки после
чисел
        String s = sc.nextLine();

        System.out.println("Вы ввели:");
        System.out.println("Целое число: " + x);
        System.out.println("Вещественное число: " + y);
        System.out.println("Строка: " + s);

        sc.close();
    }
}

```

4. Важные моменты

- **Заккрытие сканера:** После использования объекта Scanner рекомендуется закрыть его с помощью метода `close()`, чтобы избежать утечек ресурсов.

- `sc.close();`

- **Считывание строки после чисел:** если после считывания чисел требуется считать строку, необходимо сначала считать символ новой строки, оставшийся после ввода чисел:

- `sc.nextLine();`

4 Строки в Java

1. Основные понятия

Строка — это объект, представляющий последовательность символов. В Java базовый класс для строк — `String`. Например, `"Hello"` — это строка из 5 символов.

Пример объявления строки:

```
String = "Я изучаю Java";
```

Пустая строка:

```
String str = "";
```

2. Неизменяемость строк

Строки в Java являются **неизменяемыми** (immutable). Это означает, что после создания строки её нельзя изменить. Любые операции, которые изменяют строку, создают новую строку. Для работы с изменяемыми строками используется класс `StringBuilder`.

3. Конкатенация строк

Конкатенация — это сложение строк. В Java конкатенация выполняется с помощью оператора `+`.

Пример конкатенации:

```
String firstWord = "Hello";
String secondWord = "World";
System.out.println(firstWord + " " + secondWord + "!"); // Hello World!
```

4. Методы класса `String`

Класс `String` предоставляет множество методов для работы со строками. Рассмотрим некоторые из них.

4.1. Длина строки

Метод `length()`: Возвращает длину строки (количество символов).

```
String word = "Java is strong";
int x = word.length();
System.out.println(x); // 14
```

4.2. Сравнение строк

Метод `equals()`: Сравнивает строки на равенство.

```
String word1 = "Java";
String word2 = "Python";
System.out.println(word1.equals(word2)); // false
```

```
String word3 = "Ja";
String word4 = "va";
boolean result = word1.equals(word3 + word4);
System.out.println(result); // true
```

4.3. Получение индекса элемента

Метод `indexOf()`: Возвращает индекс первого вхождения символа или подстроки. Если элемент не найден, возвращает `-1`.

```
String word = "abracadabra";
int x = word.indexOf('b');
System.out.println(x); // 1
```

```
int y = word.indexOf('Z');
System.out.println(y); // -1
```

4.4. Получение элемента по индексу

Метод `charAt()`: Возвращает символ по указанному индексу.

```
String word = "abracadabra";
char letter_o = word.charAt(0);
System.out.println(letter_o); // a
```

```
char letter_4 = word.charAt(4);
```

```
System.out.println(letter_4); // с
```

4.5. Проверка строки на пустоту

Метод isEmpty(): Возвращает true, если строка пустая, и false в противном случае.

```
String str1 = "Hubba Bubba";  
String str2 = " ";  
String str3 = "";
```

```
boolean value1 = str1.isEmpty(); // false  
boolean value2 = str2.isEmpty(); // false  
boolean value3 = str3.isEmpty(); // true
```

4.6. Проверка наличия подстроки

Метод contains(): Проверяет, содержится ли одна строка внутри другой.

```
String str1 = "One Two Three";  
String str2 = "One";  
String str3 = "Four";
```

```
boolean value1 = str1.contains(str2); // true  
boolean value2 = str1.contains(str3); // false
```

4.7. Преобразование регистров

Методы toUpperCase() и toLowerCase(): Преобразуют строку в верхний и нижний регистр СООТВЕТСТВЕННО.

```
String s = "I'll be back";  
System.out.println(s.toLowerCase()); // i'll be back  
System.out.println(s.toUpperCase()); // I'LL BE BACK
```

4.8. Преобразование числа в строку

Метод toString(): Преобразует число в строку.

```
int n = 12345;  
String str1 = Integer.toString(n);  
System.out.println(str1); // 12345
```

```
Integer num = n;  
String str2 = num.toString();  
System.out.println(str2); // 12345
```

4.9. Преобразование строки в число

Методы valueOf() и parseInt(): Преобразуют строку в число.

```
String str = "12345";  
Integer num = Integer.valueOf(str);  
System.out.println(num); // 12345
```

```
int num1 = Integer.parseInt(str);  
System.out.println(num1); // 12345
```

```
double num2 = Double.parseDouble(str);  
System.out.println(num2); // 12345.0
```

4.10. Создание подстроки

Метод substring(): Возвращает подстроку, начиная с указанного индекса.

```
String str = "Добро пожаловать в мир Java!";  
System.out.println(str.substring(6)); // пожаловать в мир Java!  
System.out.println(str.substring(6, 15)); // пожаловат
```

4.11. Замена элементов строки

Метод replace(): Заменяет все вхождения одного символа на другой.

```
String str = "Добро пожаловать в мир Java!";  
System.out.println(str.replace('o', 'A')); // ДАбрА пАжалАвать в мир Java!
```

5. Вывод нескольких переменных

Java позволяет выводить на печать несколько переменных одновременно с помощью конкатенации.

Пример:

```
int x = 42;  
System.out.println("Результат вычислений равен " + x); // Результат вычислений равен 42  
  
int y = 15;  
System.out.println(x + 1 + " не равно " + y); // 43 не равно 15  
System.out.println(x + " не равно " + y + 1); // 42 не равно 151  
System.out.println(x + " не равно " + (y + 1)); // 42 не равно 16
```

5 Условные конструкции в Java

1. Оператор if

Оператор if используется для выполнения блока кода, если заданное условие истинно (true).

Синтаксис:

```
if (условие) {  
    // блок команд  
}
```

Пример:

```
int x = 7;  
if (x < 42) {  
    System.out.println("Меньше");  
}
```

Операторы сравнения:

- < — меньше
- > — больше
- == — равно
- != — не равно
- <= — меньше или равно
- >= — больше или равно

2. Оператор else

Оператор else используется вместе с оператором if для выполнения блока кода, если условие ложно (false).

Синтаксис:

```
if (условие) {  
    // блок команд, если условие истинно  
} else {  
    // блок команд, если условие ложно  
}
```

Пример:

```
int month = 10;  
if (month <= 6) {  
    System.out.println("Первая половина года");  
} else {  
    System.out.println("Вторая половина года");  
}
```

3. Тернарный оператор

Тернарный оператор — это краткая альтернатива конструкции if/else. Он позволяет выполнить проверку условия и вернуть одно из двух значений в зависимости от результата проверки.

Синтаксис:

```
условие ? выражение1 : выражение2;
```

- Если условие истинно, выполняется выражение1.
- Если условие ложно, выполняется выражение2.

Пример:

```
int x = 15;  
System.out.println((x % 2 == 0) ? "Число чётное" : "Число нечётное");
```

4. Примеры использования

Проверка чётности числа:

```
int x = 24;  
  
if (x % 2 == 0) {  
    System.out.println("Число чётное");  
}
```

```
} else {  
    System.out.println("Число нечётное");  
}
```

```
// Число чётное
```

То же самое с использованием тернарного оператора:

```
int x = 15;  
System.out.println((x % 2 == 0) ? "Число чётное" : "Число нечётное");
```

5. Важные моменты

- **Операторы сравнения:** Используйте правильные операторы сравнения (`==`, `!=`, `<`, `>`, `<=`, `>=`).
- **Блоки кода:** Блоки кода внутри `if` и `else` должны быть заключены в фигурные скобки `{}`.
- **Тернарный оператор:** Используйте его для простых условий, чтобы не усложнять код.

6 Логические конструкции в Java

1. Логические операторы

Логические операторы используются для объединения нескольких условий в одном выражении. Основные логические операторы в Java:

- **Логическое И (&&)**
- **Логическое ИЛИ (||)**
- **Логическое НЕ (!)**

2. Логическое И (&&)

Логическое И (&&) возвращает true, только если оба условия истинны. Если хотя бы одно из условий ложно, результат будет false.

Пример:

```
int age = 25;
int tasks = 600;

if (age > 18 && tasks > 500) {
    System.out.println("Добро пожаловать!");
}
```

В этом примере программа выведет "Добро пожаловать!", только если age больше 18 и tasks больше 500.

3. Логическое ИЛИ (||)

Логическое ИЛИ (||) возвращает true, если хотя бы одно из условий истинно. Если оба условия ложны, результат будет false.

Пример:

```
int age = 25;
int tasks = 100;

if (age > 18 || tasks > 500) {
    System.out.println("Добро пожаловать!");
}
```

В этом примере программа выведет "Добро пожаловать!", так как хотя бы одно из условий (age > 18) истинно.

4. Логическое НЕ (!)

Логическое НЕ (!) используется для инверсии логического значения условия. Если условие истинно, ! сделает его ложным, и наоборот.

Пример:

```
int age = 25;

if (!(age > 17)) {
    System.out.println("Вы несовершеннолетний");
} else {
    System.out.println("Добро пожаловать!");
}
```

В этом примере программа выведет "Добро пожаловать!", так как !(age > 17) эквивалентно age <= 17, что ложно для age = 25.

5. Комбинирование логических операторов

Логические операторы можно комбинировать для создания более сложных условий.

Пример:

```
int age = 25;
int tasks = 600;
boolean isAdmin = true;

if ((age > 18 && tasks > 500) || isAdmin) {
    System.out.println("Добро пожаловать!");
}
```

```
}
```

В этом примере программа выведет "Добро пожаловать!", если age больше 18 и tasks больше 500, или если пользователь является администратором (isAdmin равно true).

6. Важные моменты

- **Приоритет операторов:** Операторы && имеют более высокий приоритет, чем ||. Для изменения порядка выполнения можно использовать скобки.
- **Короткое замыкание:** В Java логические операторы && и || используют короткое замыкание. Это означает, что если результат условия можно определить по первому операнду, второй операнд не будет вычисляться.

7 Оператор switch в Java

1. Основные понятия

Оператор switch — это конструкция, которая позволяет выбирать действия в зависимости от значения переменной или выражения. Он проверяет переменную на равенство с заранее определенным списком значений и выполняет соответствующий блок кода.

2. Синтаксис

```
switch (выражение) {  
    case value1:  
        // Группа команд  
        break;  
  
    case value2:  
        // Группа команд  
        break;  
  
    // Можно вставить столько проверяемых значений, сколько нужно  
  
    default: // Необязательно  
        // Группа команд, исполняемая, если значение выражения не равно ни одному из случаев  
}
```

3. Пример использования

Пример:

```
int day = 2;  
  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
  
    case 2:  
        System.out.println("Tuesday");  
        break;  
  
    case 3:  
        System.out.println("Wednesday");  
        break;  
  
    default:  
        System.out.println("Wrong day's number");  
}
```

В этом примере программа выведет "Tuesday", так как значение переменной day равно 2.

4. Важные моменты

1. **Соответствие значений:** Чтобы исполнился определенный блок команд, значение выражения должно быть равно указанному в одном из случаев (case).
2. **Команда break:** Блок команд в каждом случае завершается командой break. Эта команда завершает исполнение всей конструкции switch, и начинается исполнение кода, расположенного после неё. Если break отсутствует, выполнится и следующий блок команд.
3. **Блок default:** Использование команды default не является обязательным. Если switch не нашел соответствия выражения ни одному из случаев, программа просто перейдет к исполнению кода, расположенного ниже.

5. Блок default

Блок default — это необязательный блок, который выполняется, если значение выражения не совпадает ни с одним из перечисленных case.

Пример:

```
int day = 7;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    case 3:
        System.out.println("Wednesday");
        break;

    default:
        System.out.println("Wrong day's number");
}
```

В этом примере программа выведет "Wrong day's number", так как значение переменной day равно 7, что не соответствует ни одному из case.

8 Цикл while в Java

1. Основные понятия

Цикл — это фундаментальная концепция программирования, которая позволяет выполнять блок команд многократно без дублирования кода. Java предоставляет несколько видов циклических конструкций, включая while, do-while, for и другие.

2. Цикл while

Цикл while исполняет блок команд (тело цикла) до тех пор, пока истинно указанное в цикле условие.

Синтаксис:

```
while (условие) {  
    // тело цикла  
}
```

Пример:

```
int x = 3;  
  
while (x > 0) {  
    System.out.println(x);  
    x--;  
}
```

Важные моменты:

- **Условие:** Условие должно возвращать логическое значение (true или false).
- **Тело цикла:** Блок команд, который выполняется, пока условие истинно.
- **Условие выхода:** Необходимо предусмотреть условие выхода из цикла, иначе код будет исполняться бесконечно.

3. Цикл do-while

Цикл do-while отличается от while тем, что проверка условия происходит не в начале, а в конце тела цикла. Таким образом, тело цикла исполняется, как минимум, один раз.

Синтаксис:

```
do {  
    // тело цикла  
} while (условие);
```

Пример:

```
int x = 1;  
  
do {  
    System.out.println(x);  
    x++;  
} while (x < 5);
```

Важные моменты:

- **Гарантированное выполнение:** Тело цикла выполняется хотя бы один раз, даже если условие ложно.
- **Точка с запятой:** После условия цикла обязательно ставится точка с запятой.

4. Команды break и continue

Команда break немедленно прерывает работу цикла, и программа переходит к командам, расположенным после цикла.

Команда continue завершает текущую итерацию цикла, и цикл переходит к исполнению следующей итерации.

Пример:

```
int x = 0;  
  
while (true) {  
    x++;  
    if (x % 2 == 0) {
```

```
        continue;
    }
    System.out.println(x);
    if (x == 5) {
        break;
    }
}
System.out.println("The end");
```

Результат:

```
1
3
5
The end
```

Важные моменты:

- **break:** Прерывает цикл и переходит к командам после цикла.
- **continue:** Завершает текущую итерацию и переходит к следующей итерации цикла.

9 Цикл for в Java

1. Основные понятия

Цикл for — это циклическая конструкция, которая используется, когда известно точное количество повторений цикла. Он более удобен, чем while, когда требуется выполнить блок команд определенное количество раз.

2. Синтаксис

```
for (инициализация; условие; счётчик) {  
    // тело цикла  
}
```

- **Инициализация:** Действие, которое выполняется только один раз в начале работы цикла. Обычно здесь задается внутренняя переменная, которая служит счетчиком количества повторений.
- **Условие:** Логическое выражение, которое вычисляется каждый раз перед итерацией цикла. Если выражение ложно, выполнение цикла прекращается.
- **Счётчик:** Выражение, которое вычисляется после каждой итерации цикла.
- **Тело цикла:** Команды, которые исполняются при каждой итерации цикла.

3. Примеры использования

Пример 1: Печать чисел от 1 до 5:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Пример 2: Печать чисел с шагом 2:

```
for (int i = 5; i < 10; i += 2) {  
    System.out.print(i + " "); // 5 7 9  
}
```

Пример 3: Печать чисел в обратном порядке:

```
for (int i = 5; i > -1; i--) {  
    System.out.print(i + " "); // 5 4 3 2 1 0  
}
```

Пример 4: Бесконечный цикл (не рекомендуется):

```
for (int i = 1; i < 10;) {  
    System.out.print(i + " "); // Бесконечный цикл  
}
```

4. Вложенные циклы

Циклы можно исполнять друг внутри друга. Вложенные циклы позволяют выполнять более сложные задачи, но их использование должно быть оправдано.

Пример: Печать букв строки по определенному правилу:

```
String s = "Java";  
  
for (int i = 0; i < s.length(); i++) {  
    if (i % 2 == 0) {  
        for (int j = 0; j < 5; j++) {  
            System.out.print(s.charAt(i));  
        }  
    } else {  
        for (int j = 0; j < 3; j++) {  
            System.out.print(s.charAt(i));  
        }  
    }  
    System.out.println(); // Перевод строки  
}
```

Результат:

```
JJJJ  
aaa
```

vvvvv
aaa

5. Важные моменты

- **Инициализация:** Выполняется только один раз в начале цикла.
- **Условие:** Проверяется перед каждой итерацией. Если условие ложно, цикл завершается.
- **Счётчик:** Выполняется после каждой итерации.
- **Бесконечные циклы:** Избегайте бесконечных циклов, так как они могут привести к зависанию программы.
- **Вложенные циклы:** Используйте их с осторожностью, чтобы не усложнить код.

10 Массивы в Java

1. Основные понятия

Массив — это набор переменных одного типа, которые хранятся упорядоченно.

Массивы позволяют хранить список значений, таких как числа, строки и другие типы данных.

2. Объявление и инициализация массива

Объявление массива:

```
int[] arr;
```

Создание массива с указанием размера:

```
int[] arr = new int[5];
```

Инициализация массива с конкретными значениями:

```
String[] names = {"A", "B", "C", "D"};
```

Альтернативный способ объявления массива:

```
String names[] = {"A", "B", "C", "D"};
```

3. Доступ к элементам массива

Индексация:

- Индексация элементов в массиве начинается с нуля.
- Первый элемент имеет индекс 0, а последний — length - 1.

Пример:

```
int[] arr = new int[5];
arr[2] = 42;

System.out.println(arr[2]); // 42
System.out.println(arr[3]); // 0
```

4. Длина массива

Получение длины массива:

```
int[] newArray = new int[5];
System.out.println(newArray.length); // 5
```

5. Создание массива через цикл

Пример: Создание массива натуральных чисел от 10 до 20:

```
int[] numbers = new int[11];

for (int i = 10; i <= 20; i++) {
    numbers[i - 10] = i;
}
```

6. Обработка массива

Пример: Умножение всех элементов массива на 2:

```
for (int i = 0; i < numbers.length; i++) {
    numbers[i] = numbers[i] * 2;
}
```

Пример: Вывод всех элементов массива:

```
for (int i = 0; i < numbers.length; i++) {
    System.out.print(numbers[i] + " ");
}
```

7. Сокращенный цикл for-each

Пример: Обойти все элементы массива:

```
String[] beatles = {"John", "Paul", "Ringo", "George"};

for (String person : beatles) {
    System.out.println(person);
}
```

8. Построение массива из строки

Метод split() класса String:

```
String str = "I'll be back";  
String[] words = str.split(" ");  
for (String word : words) {  
    System.out.println(word);  
}
```

Пример: Разделение строки по символу ;:

```
String dataLine = "158;190;178;186;174";  
String[] dataHeight = dataLine.split(";");  
  
for (String el : dataHeight) {  
    System.out.println(el);  
}
```

9. Важные моменты

- **Индексация:** Начинается с 0.
- **Длина массива:** Получается через атрибут `length`.
- **Сокращенный цикл for-each:** Удобен для обхода всех элементов массива.
- **Метод `split()`:** Разделяет строку на массив строк по указанному разделителю.

11 Методы класса Arrays в Java

Для работы с массивами в Java существует специальный класс Arrays, который предоставляет множество полезных методов. Рассмотрим некоторые из них.

1. Импорт класса Arrays

Перед использованием методов класса Arrays необходимо импортировать его:

```
import java.util.Arrays;
```

2. Метод Arrays.toString()

Описание: Преобразует массив в строку. Полезен для вывода массива на печать.

Пример:

```
int[] array = {1, 2, 3, 4, 5};  
System.out.println(Arrays.toString(array)); // [1, 2, 3, 4, 5]
```

3. Метод Arrays.sort()

Описание: Сортирует массив или его часть.

Пример:

```
int[] array = {5, 10, 15, 4, 1};  
Arrays.sort(array);  
System.out.println(Arrays.toString(array)); // [1, 4, 5, 10, 15]
```

Сортировка части массива:

```
int[] array = {5, 10, 15, 4, 1, 7, 23, 8, 11, 42};  
Arrays.sort(array, 3, 7);  
System.out.println(Arrays.toString(array)); // [5, 10, 15, 1, 4, 7, 23, 8, 11, 42]
```

Сортировка массива строк:

```
String[] array = {"one", "two", "three", "four"};  
Arrays.sort(array);  
System.out.println(Arrays.toString(array)); // [four, one, three, two]
```

4. Метод Arrays.equals()

Описание: Сравнивает два массива и возвращает true, если они равны, и false — в обратном случае.

Пример:

```
int[] numbers1 = {1, 2, 3};  
int[] numbers2 = {1, 2, 3};  
  
System.out.println(Arrays.equals(numbers1, numbers2)); // true  
  
int[] numbers3 = {1, 2, 3, 4};  
boolean result = Arrays.equals(numbers1, numbers3);  
System.out.println(result); // false
```

Важно: Сравнивать массивы нужно с помощью метода Arrays.equals(), а не через equals() экземпляра массива.

5. Метод Arrays.copyOf()

Описание: Возвращает копию массива. Метод получает в качестве аргументов массив, который нужно скопировать, и число элементов, которые нужно скопировать.

Пример:

```
int[] numbers1 = {1, 2, 3, 4, 5};  
int[] numbers2 = Arrays.copyOf(numbers1, 5);  
System.out.println(Arrays.toString(numbers2)); // [1, 2, 3, 4, 5]  
  
int[] numbers3 = Arrays.copyOf(numbers1, 3);  
System.out.println(Arrays.toString(numbers3)); // [1, 2, 3]
```

Копирование всего массива:

```
int[] numbers1 = {1, 2, 3, 4, 5};  
int[] numbers2 = Arrays.copyOf(numbers1, numbers1.length);  
System.out.println(Arrays.toString(numbers2)); // [1, 2, 3, 4, 5]
```

Важно: Приравнивание массивов (`numbers2 = numbers1`) не создает копию, а лишь ссылку на тот же объект.

6. Метод `Arrays.copyOfRange()`

Описание: Возвращает копию части массива. Метод получает на вход массив, а также стартовый и конечный индексы "куска", который нужно скопировать.

Пример:

```
int[] numbers1 = {2, 4, 6, 8, 10, 12, 14, 16};
int[] numbers2 = Arrays.copyOfRange(numbers1, 2, 5);
System.out.println(Arrays.toString(numbers2)); // [6, 8, 10]
```

12 Цикл с обходом элементов массива в Java

1. Обычный цикл `for`

Часто требуется обойти весь массив и выполнить какую-либо операцию над каждым элементом. Например, вывести на печать все строки, начинающиеся на букву `f`:

```
String[] numbers = {"one", "two", "three", "four", "five", "forty two"};

for (int i = 0; i < numbers.length; i++) {
    if (numbers[i].charAt(0) == 'f') {
        System.out.println(numbers[i]);
    }
}
```

2. Цикл с обходом элементов (`enhanced for loop`)

В Java есть специальный цикл, который упрощает обход всех элементов массива. Этот цикл называется **enhanced for loop** или **for-each loop**.

Синтаксис:

```
for (Тип_элемента переменная : массив) {
    // тело цикла
}
```

Пример:

```
String[] numbers = {"one", "two", "three", "four", "five", "forty two"};

for (String s : numbers) {
    if (s.charAt(0) == 'f') {
        System.out.println(s);
    }
}
```

3. Важные моменты

- **Инициализация переменной:** Внутри цикла инициализируется переменная, соответствующая типу элементов массива. Эта переменная в каждой итерации цикла будет равна соответствующему элементу массива.
- **Двоеточие:** Переменная отделяется от массива двоеточием (`:`).
- **Упрощение кода:** Использование такого цикла упрощает код и повышает его понятность.

4. Разница между обычным циклом и `for-each`

Обычный цикл `for`:

```
String[] numbers = {"one", "two", "three", "four", "five", "forty two"};

for (int i = 0; i < numbers.length; i++) {
    if (numbers[i].charAt(0) == 'f') {
        numbers[i] = "deleted";
    }
}

System.out.println(numbers[3]); // deleted
```

Цикл `for-each`:

```
String[] numbers1 = {"one", "two", "three", "four", "five", "forty two"};

for (String s : numbers1) {
    if (s.charAt(0) == 'f') {
        s = "deleted";
    }
}

System.out.println(numbers1[3]); // four
```

Важно: В цикле for-each переменная *s* является копией элемента массива, поэтому изменения, внесенные в *s*, не влияют на оригинальный массив. В обычном цикле *for* мы работаем непосредственно с элементами массива, поэтому изменения отражаются на оригинальном массиве.

13 Многомерные массивы в Java

1. Основные понятия

Многомерный массив (или вложенный массив) — это массив, каждый элемент которого, в свою очередь, является массивом. Многомерные массивы часто используются для представления таблиц, матриц и других структур данных, где требуется более сложная организация данных.

2. Объявление многомерного массива

Пример объявления двумерного массива:

```
int[][] sample = {{1, 2, 3}, {4, 5, 6}};
```

В этом примере объявлен двумерный массив `sample`, который содержит два элемента: `{1, 2, 3}` и `{4, 5, 6}`. Каждый элемент также является массивом.

3. Доступ к элементам многомерного массива

Доступ к элементам:

```
int[][] sample = {{1, 2, 3}, {4, 5, 6}};
```

```
int[] first = sample[0]; // {1, 2, 3}
```

```
int num = sample[0][1];
```

```
System.out.println(num); // 2
```

```
System.out.println(sample[1][2]); // 6
```

Важные моменты:

- Индексация начинается с нуля.
- Если указать неверный индекс, программа выдаст ошибку.

4. Элементы разной длины

Элементы многомерного массива могут быть разной длины:

```
int[][] myArr = {{1, 2, 3}, {4}, {5, 6, 7, 8}};
```

```
System.out.println(myArr[1][0]); // 4
```

В этом примере определен двумерный массив (таблица), состоящий из трех "строк", при этом в первой "строке" — три "ячейки", во второй — одна, а в третьей — четыре.

5. Изменение значений в многомерном массиве

Изменение значений:

```
int[][] myArr = {{1, 2, 3}, {4}, {5, 6, 7, 8}};
```

```
System.out.println(myArr[2][2]); // 7
```

```
myArr[2][2] = 42;
```

```
System.out.println(myArr[2][2]); // 42
```

6. Многомерные массивы с большей вложенностью

Java не ограничивает количество уровней вложенности в многомерных массивах.

Однако нужно помнить, что управлять многомерными массивами с большой вложенностью очень сложно.

Пример трехмерного массива:

```
int[][][] threeDArray = {  
    {{1, 2}, {3, 4}},  
    {{5, 6}, {7, 8}}  
};
```

```
System.out.println(threeDArray[1][0][1]); // 6
```

14 Интерфейс List в Java

1. Основные понятия

Интерфейс List — это подтип интерфейса Collection, который представляет упорядоченную коллекцию элементов. List позволяет дублировать элементы и поддерживает доступ к элементам по индексу.

2. Основные реализации List

2.1. ArrayList

Описание:

- Реализует интерфейс List.
- Хранит элементы в динамическом массиве.
- Обеспечивает быстрый доступ по индексу.
- Подходит для случаев, когда требуется частое обращение к элементам по индексу.

Пример:

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        System.out.println(list); // [Apple, Banana, Cherry]
    }
}
```

2.2. LinkedList

Описание:

- Реализует интерфейсы List и Deque.
- Хранит элементы в виде двусвязного списка.
- Обеспечивает быструю вставку и удаление элементов.
- Подходит для случаев, когда требуется частая вставка и удаление элементов.

Пример:

```
import java.util.LinkedList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> list = new LinkedList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        System.out.println(list); // [Apple, Banana, Cherry]
    }
}
```

3. Основные методы интерфейса List

3.1. Добавление элементов

add(E e)

- Добавляет элемент в конец списка.

Пример:

```
list.add("Date");
System.out.println(list); // [Apple, Banana, Cherry, Date]
```

add(int index, E element)

- Вставляет элемент на указанную позицию.

Пример:

```
list.add(1, "Grape");  
System.out.println(list); // [Apple, Grape, Banana, Cherry, Date]
```

3.2. Удаление элементов

remove(int index)

- Удаляет элемент по указанному индексу.

Пример:

```
list.remove(2);  
System.out.println(list); // [Apple, Grape, Cherry, Date]
```

remove(Object o)

- Удаляет первое вхождение указанного элемента.

Пример:

```
list.remove("Grape");  
System.out.println(list); // [Apple, Cherry, Date]
```

3.3. Получение элементов

get(int index)

- Возвращает элемент по указанному индексу.

Пример:

```
String fruit = list.get(1);  
System.out.println(fruit); // Cherry
```

3.4. Проверка наличия элементов

contains(Object o)

- Проверяет, содержится ли указанный элемент в списке.

Пример:

```
boolean containsBanana = list.contains("Banana");  
System.out.println(containsBanana); // false
```

3.5. Размер списка

size()

- Возвращает количество элементов в списке.

Пример:

```
int size = list.size();  
System.out.println(size); // 3
```

3.6. Очистка списка

clear()

- Удаляет все элементы из списка.

Пример:

```
list.clear();  
System.out.println(list); // []
```

3.7. Обход элементов

Цикл for-each:

```
for (String fruit : list) {  
    System.out.println(fruit);  
}
```

Итератор:

```
import java.util.Iterator;  
  
Iterator<String> iterator = list.iterator();  
while (iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

4. Важные моменты

- **Индексация:** Индексация элементов в List начинается с нуля.

- **Типизация:** List поддерживает типизацию с использованием дженериков (<T>).
- **Производительность:** ArrayList обеспечивает быстрый доступ по индексу, а LinkedList — быструю вставку и удаление элементов.

15 Функции и методы в Java

1. Основные понятия

Метод — это блок кода, который выполняет определенную задачу. Методы позволяют организовать код в логические блоки, что делает его более читаемым и повторно используемым. В Java все методы должны быть объявлены внутри класса.

2. Синтаксис метода

Синтаксис:

```
[модификаторы] тип_возвращаемого_значения имя_метода([параметры]) {  
    // тело метода  
    return значение; // если метод возвращает значение  
}
```

- **Модификаторы:** Определяют видимость и доступность метода (например, public, private, static).
- **Тип возвращаемого значения:** Указывает тип данных, который метод возвращает. Если метод не возвращает значение, используется ключевое слово void.
- **Имя метода:** Уникальное имя метода.
- **Параметры:** Переменные, которые передаются в метод.
- **Тело метода:** Блок кода, который выполняется при вызове метода.
- **return:** Возвращает значение из метода. Если метод возвращает void, ключевое слово return может быть опущено.

3. Примеры методов

3.1. Метод без параметров и возвращаемого значения

Пример:

```
public class Main {  
    public static void main(String[] args) {  
        greet();  
    }  
  
    public static void greet() {  
        System.out.println("Hello, World!");  
    }  
}
```

Объяснение:

- **public static void greet():** Объявление метода greet, который не принимает параметров и не возвращает значение (void).
- **System.out.println("Hello, World!");** Вывод строки "Hello, World!" на консоль.
- **greet();** Вызов метода greet из метода main.

3.2. Метод с параметрами и возвращаемым значением

Пример:

```
public class Main {  
    public static void main(String[] args) {  
        int result = add(5, 3);  
        System.out.println("Result: " + result);  
    }  
  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}
```

Объяснение:

- **public static int add(int a, int b):** Объявление метода add, который принимает два параметра (int a и int b) и возвращает значение типа int.

- **return a + b;** Возвращает сумму параметров a и b.
- **int result = add(5, 3);** Вызов метода add с аргументами 5 и 3, результат сохраняется в переменную result.
- **System.out.println("Result: " + result);** Вывод результата на консоль.

3.3. Метод с переменным количеством аргументов (varargs)

Пример:

```
public class Main {
    public static void main(String[] args) {
        int sum = sum(1, 2, 3, 4, 5);
        System.out.println("Sum: " + sum);
    }

    public static int sum(int... numbers) {
        int total = 0;
        for (int number : numbers) {
            total += number;
        }
        return total;
    }
}
```

Объяснение:

- **public static int sum(int... numbers):** Объявление метода sum, который принимает переменное количество аргументов типа int (varargs).
- **int total = 0;** Инициализация переменной total для хранения суммы.
- **for (int number : numbers):** Цикл for-each для обхода всех элементов массива numbers.
- **total += number;** Суммирование элементов.
- **return total;** Возвращает сумму элементов.
- **int sum = sum(1, 2, 3, 4, 5);** Вызов метода sum с переменным количеством аргументов.
- **System.out.println("Sum: " + sum);** Вывод результата на консоль.

4. Перегрузка методов

Перегрузка методов — это возможность объявлять несколько методов с одинаковым именем, но разными параметрами. Java определяет, какой метод вызвать, основываясь на типе и количестве аргументов.

Пример:

```
public class Main {
    public static void main(String[] args) {
        System.out.println(multiply(2, 3)); // 6
        System.out.println(multiply(2.5, 3.5)); // 8.75
    }

    public static int multiply(int a, int b) {
        return a * b;
    }

    public static double multiply(double a, double b) {
        return a * b;
    }
}
```

Объяснение:

- **public static int multiply(int a, int b):** Объявление метода multiply, который принимает два параметра типа int и возвращает значение типа int.
- **public static double multiply(double a, double b):** Объявление метода multiply, который принимает два параметра типа double и возвращает значение типа double.

- `System.out.println(multiply(2, 3));`: Вызов метода `multiply` с аргументами типа `int`.
- `System.out.println(multiply(2.5, 3.5));`: Вызов метода `multiply` с аргументами типа `double`.

5. Область видимости переменных

Область видимости переменных — это часть кода, где переменная доступна для использования. В Java переменные могут иметь разные области видимости:

- **Локальные переменные:** Объявляются внутри метода и доступны только в пределах этого метода.
- **Переменные экземпляра:** Объявляются внутри класса, но вне методов. Доступны для всех методов класса.
- **Статические переменные:** Объявляются с ключевым словом `static` и доступны для всех экземпляров класса.

Пример:

```
public class Main {
    static int staticVar = 10; // Статическая переменная

    public static void main(String[] args) {
        int localVar = 5; // Локальная переменная
        System.out.println("Local variable: " + localVar);
        System.out.println("Static variable: " + staticVar);
    }
}
```

Объяснение:

- `static int staticVar = 10;`: Объявление статической переменной `staticVar`, которая доступна для всех экземпляров класса.
- `int localVar = 5;`: Объявление локальной переменной `localVar`, которая доступна только в пределах метода `main`.
- `System.out.println("Local variable: " + localVar);`: Вывод значения локальной переменной на консоль.
- `System.out.println("Static variable: " + staticVar);`: Вывод значения статической переменной на консоль.

6. Важные моменты

- **Методы и классы:** В Java все методы должны быть объявлены внутри класса.
- **Перегрузка методов:** Позволяет использовать одно и то же имя метода с разными параметрами.
- **Область видимости:** Определяет, где переменная доступна для использования.
- **Статические методы:** Методы, которые принадлежат классу, а не экземпляру класса.

16 Объектно-ориентированное программирование (ООП) в Java

1. Основные понятия

Объектно-ориентированное программирование (ООП) — это парадигма программирования, которая организует программу вокруг объектов, а не функций и логики. Основные принципы ООП включают:

- **Классы и объекты:** Класс — это шаблон или описание для создания объектов. Объект — это экземпляр класса.
- **Наследование:** Возможность создания нового класса на основе существующего, наследуя его свойства и методы.
- **Инкапсуляция:** Скрытие внутренних деталей реализации и предоставление доступа только через публичные методы.
- **Полиморфизм:** Возможность использовать один и тот же интерфейс для разных типов данных.

2. Классы и объекты

2.1. Объявление класса

Синтаксис:

```
[модификаторы] class ИмяКласса {  
    // поля (переменные экземпляра)  
    // конструкторы  
    // методы  
}
```

Пример:

```
public class Car {  
    // Поля (переменные экземпляра)  
    String brand;  
    String model;  
    int year;  
  
    // Конструктор  
    public Car(String brand, String model, int year) {  
        this.brand = brand;  
        this.model = model;  
        this.year = year;  
    }  
  
    // Методы  
    public void start() {  
        System.out.println("Car started");  
    }  
  
    public void stop() {  
        System.out.println("Car stopped");  
    }  
}
```

2.2. Создание объекта

Синтаксис:

```
ИмяКласса имяОбъекта = new ИмяКласса([параметры]);
```

Пример:

```
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car("Toyota", "Camry", 2020);  
        myCar.start();  
        myCar.stop();  
    }  
}
```

3. Наследование

Наследование — это механизм, который позволяет создавать новый класс на основе существующего, наследуя его свойства и методы.

Синтаксис:

```
class Подкласс extends Суперкласс {  
    // дополнительные поля и методы  
}
```

Пример:

```
public class Vehicle {  
    String brand;  
  
    public Vehicle(String brand) {  
        this.brand = brand;  
    }  
  
    public void move() {  
        System.out.println("Vehicle is moving");  
    }  
}  
  
public class Car extends Vehicle {  
    String model;  
  
    public Car(String brand, String model) {  
        super(brand);  
        this.model = model;  
    }  
  
    @Override  
    public void move() {  
        System.out.println("Car is moving");  
    }  
}
```

Объяснение:

- **class Car extends Vehicle:** Класс Car наследует класс Vehicle.
- **super(brand):** Вызов конструктора суперкласса.
- **@Override:** Аннотация, указывающая, что метод переопределен.

4. Инкапсуляция

Инкапсуляция — это механизм, который скрывает внутренние детали реализации и предоставляет доступ только через публичные методы.

Синтаксис:

```
[модификаторы] тип_данных имя_поля;
```

Пример:

```
public class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
        }  
    }  
}
```

```
public double getBalance() {  
    return balance;  
}
```

Объяснение:

- **private double balance:** Поле balance скрыто от прямого доступа извне.
- **public void deposit(double amount):** Публичный метод для внесения денег.
- **public void withdraw(double amount):** Публичный метод для снятия денег.
- **public double getBalance():** Публичный метод для получения баланса.

5. Полиморфизм

Полиморфизм — это возможность использовать один и тот же интерфейс для разных типов данных.

5.1. Переопределение методов

Пример:

```
public class Animal {  
    public void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
public class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Cat extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Cat meows");  
    }  
}
```

Объяснение:

- **@Override:** Аннотация, указывающая, что метод переопределен.
- **makeSound():** Метод, который переопределяется в подклассах.

5.2. Полиморфизм в действии

Пример:

```
public class Main {  
    public static void main(String[] args) {  
        Animal myDog = new Dog();  
        Animal myCat = new Cat();  
  
        myDog.makeSound(); // Dog barks  
        myCat.makeSound(); // Cat meows  
    }  
}
```

Объяснение:

- **Animal myDog = new Dog();** Создание объекта Dog с типом Animal.
- **Animal myCat = new Cat();** Создание объекта Cat с типом Animal.
- **myDog.makeSound();** Вызов метода makeSound() для объекта Dog.
- **myCat.makeSound();** Вызов метода makeSound() для объекта Cat.

6. Абстрактные классы и интерфейсы

6.1. Абстрактные классы

Абстрактный класс — это класс, который не может быть инстанцирован (создан объект), но может содержать абстрактные методы.

Синтаксис:

```
abstract class ИмяКласса {  
    // абстрактные методы  
}
```

Пример:

```
public abstract class Shape {  
    public abstract double getArea();  
}  
  
public class Circle extends Shape {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    @Override  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
}
```

Объяснение:

- **abstract class Shape:** Объявление абстрактного класса Shape.
- **public abstract double getArea();** Объявление абстрактного метода getArea().
- **public class Circle extends Shape:** Класс Circle наследует абстрактный класс Shape.
- **@Override:** Аннотация, указывающая, что метод переопределен.

6.2. Интерфейсы

Интерфейс — это контракт, который определяет набор методов, которые должны быть реализованы классом.

Синтаксис:

```
interface ИмяИнтерфейса {  
    // методы интерфейса  
}
```

Пример:

```
public interface Drawable {  
    void draw();  
}  
  
public class Circle implements Drawable {  
    @Override  
    public void draw() {  
        System.out.println("Drawing a circle");  
    }  
}
```

Объяснение:

- **interface Drawable:** Объявление интерфейса Drawable.
- **void draw();** Метод интерфейса, который должен быть реализован.
- **public class Circle implements Drawable:** Класс Circle реализует интерфейс Drawable.
- **@Override:** Аннотация, указывающая, что метод переопределен.

7. Важные моменты

- **Классы и объекты:** Основа ООП в Java.
- **Наследование:** Позволяет создавать новые классы на основе существующих.

- **Инкапсуляция:** Скрывает внутренние детали реализации.
- **Полиморфизм:** Позволяет использовать один и тот же интерфейс для разных типов данных.
- **Абстрактные классы и интерфейсы:** Определяют контракты для классов.